SIMATS ENGINEERING

ASSESSMENT TOOL 3 (AT3) — COURSE OUTCOME 2 (CO2)

Course Name:	Artificial Intelligence	**Course Code:**	CSA17
Student Name:	**Shaik Reehan**	**Register Number:**	**192565071**
Duration:	1 Hour	**Total Marks:**	30 (Weightage: 40%)
Course Outcome:	CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)		

Code of Conduct & Student Declaration:

I, **Shaik Reehan (Reg No: 192565071)**, certify that this submission is my original work and that I have adhered to the academic integrity guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

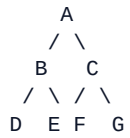
Signature of Student: Shaik Reehan **Date of Submission:** October 21, 2026

ANNEXURE A: DRY RUN EVALUATION

Question 1: Breadth-First Search (BFS) Dry Run Trace

Problem Description: Consider the following binary tree graph. Starting from root node **A**, perform a Breadth-First Search (BFS) dry run, trace the queue contents at each iteration, and determine the exact traversal order.

1. Graph Structure & Search Principles



- **Start Node:** *A* (Root).
- **Data Structure:** FIFO (First-In, First-Out) Queue.
- **Expansion Order:** Left-to-right alphabetical child expansion (*B* then *C*; *D* then *E*; *F* then *G*).

2. Completed BFS Dry Run Table

Iteration	Visited Node (Dequeued)	Queue Before Expansion	Queue After Expansion (Front → Rear)
1	**A**	[A]	[B, C]
2	**B**	[B, C]	[C, D, E]
3	**C**	[C, D, E]	[D, E, F, G]
4	**D**	[D, E, F, G]	[E, F, G]
5	**E**	[E, F, G]	[F, G]
6	**F**	[F, G]	[G]
7	**G**	[G]	[] (Empty)

3. Sub-Question Answers

1. **BFS Traversal Order:**

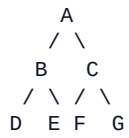
$$\boldsymbol{A \rightarrow B \rightarrow C \rightarrow D \rightarrow E \rightarrow F \rightarrow G}$$

2. **Queue Contents After Each Iteration:**
   - After Iteration 1: *[B, C]*
   - After Iteration 2: *[C, D, E]*
   - After Iteration 3: *[D, E, F, G]*
   - After Iteration 4: *[E, F, G]*
   - After Iteration 5: *[F, G]*
   - After Iteration 6: *[G]*
   - After Iteration 7: *[]* (Queue empty, search terminates)

Question 2: Depth-First Search (DFS) Dry Run Trace

Problem Description: Using the same binary tree graph, perform Depth-First Search (DFS) starting from root node **A**. Trace the LIFO stack contents across all iterations and determine the exact DFS traversal order.

1. Graph Structure & Search Principles



- **Start Node:** *A* (Root).
- **Data Structure:** LIFO (Last-In, First-Out) Stack.
- **Push Order:** Children are pushed right-to-left (push *C*, then push *B*) so that the left child (*B*) resides on top of the stack and is popped first.

2. Completed DFS Dry Run Table

Iteration	Visited Node (Popped)	Stack Before Expansion (Top → Bottom)	Stack After Expansion (Top → Bottom)
1	**A**	[A]	[B, C]
2	**B**	[B, C]	[D, E, C]
3	**D**	[D, E, C]	[E, C]
4	**E**	[E, C]	[C]
5	**C**	[C]	[F, G]
6	**F**	[F, G]	[G]
7	**G**	[G]	[] (Empty)

3. DFS Traversal Order & Algorithmic Comparison

- **DFS Traversal Order:**

$$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C \rightarrow F \rightarrow G$$

- **Detailed Walkthrough of Backtracking Mechanism:**
  1. At iteration 1, root *A* expands children *B* and *C*.
  2. At iteration 2, *B* is popped and pushes its children *E* and *D* (with *D* on top).
  3. At iteration 3, leaf node *D* is popped. Having no children, the algorithm backtracks to next stack top *E*.
  4. At iteration 4, leaf node *E* is popped. Having no children, the entire left branch under *B* is exhausted, and the search backtracks to *C*.
  5. At iteration 5, *C* is popped and expands *F* and *G*. Leaf nodes *F* and *G* are processed sequentially in iterations 6 and 7.
- **Theoretical Comparison (BFS vs. DFS):**

Metric	Breadth-First Search (BFS)	Depth-First Search (DFS)
Data Structure	FIFO Queue	LIFO Stack (or recursion stack)
Space Complexity	$O(b^d)$ — stores all active frontier nodes (high memory).	$O(b \times d)$ — linear space proportional to depth (low memory).
Time Complexity	$O(b^d)$	$O(b^m)$ (where m is maximum tree depth)
Optimality	Optimal for unweighted graphs (finds shortest path).	Not optimal (may return longer paths along deep branches).

Question 3: 8-Puzzle Problem using Breadth-First Search (BFS)

Problem Description: An 8-puzzle consists of a 3×3 board with 8 numbered tiles and an empty space (represented by 0). The objective is to transition from the Initial State to the Goal State by sliding tiles into the blank space.

1. Problem Formulation & State Configurations

Initial State (S_0)	Goal State (G)
1 3 6 5 0 2 4 7 8	1 2 3 4 5 6 7 8 0

2. BFS State Expansion Trace Table (First 5 Iterations)

Iter.	Current State Expanded	Queue Before Expansion	Queue After Expansion
1	$\mathbf{S_0}$: [1 3 6; 5 0 2; 4 7 8] (Blank at center)	[S_0]	[S_1, S_2, S_3, S_4]
2	$\mathbf{S_1}$: [1 0 6; 5 3 2; 4 7 8] (Blank moved Up)	[S_1, S_2, S_3, S_4]	[S_2, S_3, S_4, S_5, S_6]
3	$\mathbf{S_2}$: [1 3 6; 5 7 2; 4 0 8] (Blank moved Down)	[S_2, S_3, S_4, S_5, S_6]	[S_3, S_4, S_5, S_6, S_7, S_8]
4	$\mathbf{S_3}$: [1 3 6; 0 5 2; 4 7 8] (Blank moved Left)	[S_3, S_4, S_5, S_6, S_7, S_8]	[S_4, S_5, S_6, S_7, S_8, S_9, S_{10}]
5	$\mathbf{S_4}$: [1 3 6; 5 2 0; 4 7 8] (Blank moved Right)	[S_4, S_5, S_6, S_7, S_8, S_9, S_{10}]	[S_5, S_6, S_7, S_8, S_9, S_{10}, S_{11}, S_{12}]

3. Sub-Question Answers

1. **Which node is expanded in each iteration?**
   - Iteration 1: S_0 (Initial state with blank 0 at center).
   - Iteration 2: S_1 (State resulting from sliding tile 3 Down / blank Up).
   - Iteration 3: S_2 (State resulting from sliding tile 7 Up / blank Down).
   - Iteration 4: S_3 (State resulting from sliding tile 5 Right / blank Left).
   - Iteration 5: S_4 (State resulting from sliding tile 2 Left / blank Right — leads to optimal solution).
2. **How many states are generated in total?**
   - Iteration 1 generates 4 states (S_1, S_2, S_3, S_4).
   - Iteration 2 generates 2 unvisited states (S_5, S_6).
   - Iteration 3 generates 2 unvisited states (S_7, S_8).
   - Iteration 4 generates 2 unvisited states (S_9, S_{10}).
   - Iteration 5 generates 2 unvisited states (S_{11}, S_{12}).
   - **Total states generated across 5 iterations:** 1 initial state + 12 successor states = **13 states**.
3. **Complete Solution Path from Initial State to Goal State (8 Moves):**

Step 0 (Start)	Step 1 (Right)	Step 2 (Up)	Step 3 (Left)	Step 4 (Down)	Step 5 (Left)	Step 6 (Down)	Step 7 (Right)	Step 8 (Goal!)
1 3 6 5 0 2 4 7 8	1 3 6 5 2 0 4 7 8	1 3 0 5 2 6 4 7 8	1 0 3 5 2 6 4 7 8	1 2 3 5 0 6 4 7 8	1 2 3 0 5 6 4 7 8	1 2 3 4 5 6 0 7 8	1 2 3 4 5 6 7 0 8	1 2 3 4 5 6 7 8 0

   Move Sequence: *Right → Up → Left → Down → Left → Down → Right → Right* (Total cost: **8 moves**).
4. **Why does BFS always find the shortest solution in an unweighted 8-puzzle problem?**
   In an unweighted 8-puzzle problem, every tile movement has an identical unit cost ($c = 1$). Breadth-First Search systematically visits the state space level-by-level in strictly non-decreasing path length ($d = 0, 1, 2, ...$). Since all nodes at depth $d - 1$ are completely expanded before any node at depth d is visited, the very first time the Goal state is reached, it is mathematically guaranteed that no solution exists at any shallower depth $< d$. Hence, BFS is guaranteed to return the minimal-length, optimal solution path.